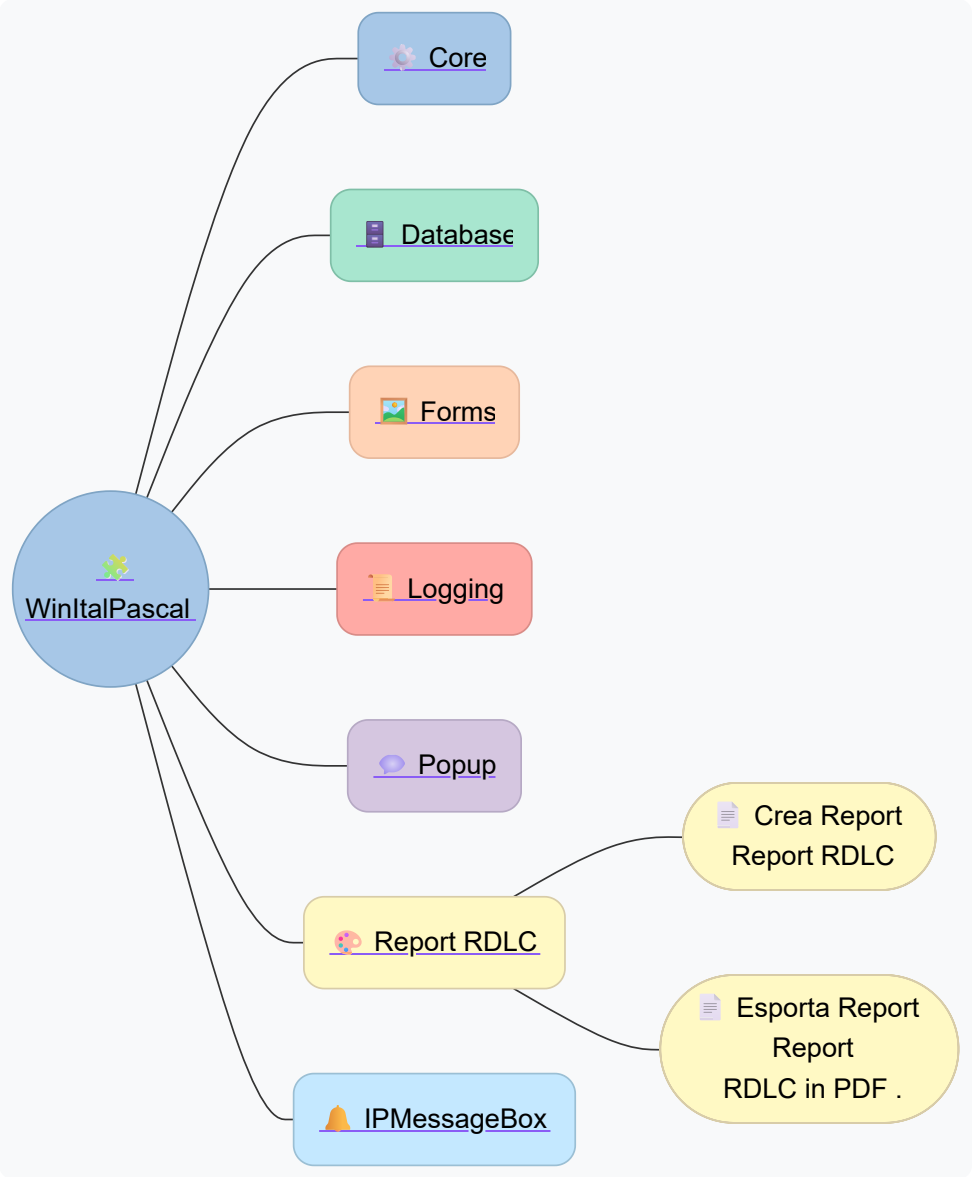
Ital Pascal


Core
Database
Forms
WinItalPascal
Logging
Popup
Crea Report
Report RDLC
Report RDLC
Esporta Report
Report
RDLC in PDF .
IPMessageBox



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Scopo generale del codice

Il modulo `ReportImpostazioni` fornisce un helper per configurare le impostazioni di stampa di un **ReportViewer** (Microsoft.Reporting.WinForms) in modo da stampare un documento in formato **A4 orizzontale** con margini personalizzati.
In pratica, permette di impostare in un'unica chiamata le proprietà di pagina (dimensione, orientamento, margini) che verranno poi applicate al controllo di visualizzazione del report.

Componenti chiave e funzioni

Elemento	Descrizione	Note
`Imports System.Drawing.Printing`	Importa il namespace che contiene le classi per la gestione delle impostazioni di stampa (`PageSettings` , `PaperSize` , `Margins`).	Necessario per accedere a queste classi.
`Imports Microsoft.Reporting.WinForms`	Importa il namespace del controllo `ReportViewer` .	Permette di accedere al tipo `ReportViewer` .
`Public Module ReportImpostazioni`	Modulo pubblico che raggruppa funzioni di utilità.	In VB.NET i moduli sono simili a classi statiche.
`#Region "Impostazione Orizzontale"`	Sezione di codice per l'organizzazione visiva.	Non influisce sull'esecuzione.
`Public Sub SetImpOrd(viewer As ReportViewer)`	Subroutine pubblica che accetta un oggetto `ReportViewer` e imposta le sue impostazioni di pagina.	Nome "SetImpOrd" è abbreviato; potrebbe essere più descrittivo.

Elemento	Descrizione	Note
`If viewer Is Nothing Then Exit Sub`	Controllo di sicurezza: se il parametro è `Nothing`, la sub termina subito.	Evita eccezioni di riferimento nullo.
`Dim ps As New PageSettings()`	Istanzia un oggetto `PageSettings` che conterrà tutte le proprietà di pagina.	
`ps.PaperSize = New PaperSize("A4", 827, 1169)`	Imposta la dimensione del foglio a A4. I valori 827 e 1169 sono in centesimi di pollice (≈ 21 cm × 29,7 cm).	
`ps.Landscape = True`	Imposta l'orientamento a orizzontale (landscape).	
`ps.Margins = New Margins(50, 50, 50, 50)`	Imposta margini di 50 centesimi di pollice (~0,5 cm) su tutti i lati.	
`viewer.SetPageSettings(ps)`	Applica le impostazioni di pagina al `ReportViewer`.	
`End Sub`	Fine della subroutine.	
`End Module`	Fine del modulo.	

Flusso di esecuzione

1. **Chiamata**: L'utente chiama `ReportImpostazioni.SetImpOrd(ReportViewer1)`.
2. **Verifica**: La subroutine controlla se l'argomento `viewer` è `Nothing`. Se lo è, termina senza fare nulla.
3. **Creazione**: Viene creato un nuovo oggetto `PageSettings`.
4. **Configurazione**:
   - Si imposta la dimensione del foglio a A4 (827 × 1169 centesimi di pollice).
   - Si imposta l'orientamento a orizzontale (`Landscape = True`).
   - Si impostano margini di 50 centesimi di pollice su tutti i lati.
5. **Applicazione**: Le impostazioni appena configurate vengono applicate al `ReportViewer` tramite `SetPageSettings`.
6. **Fine**: La subroutine termina; il `ReportViewer` ora è pronto per la stampa con le nuove impostazioni.

🔹 Conclusione

Il modulo `ReportImpostazioni` è un piccolo helper che centralizza la configurazione di stampa di un `ReportViewer` in formato A4 orizzontale con margini di 0,5 cm.
La sua struttura è semplice: verifica l'input, crea un oggetto `PageSettings`, imposta le proprietà desiderate e le applica al controllo.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

💠 Gestione centralizzata dei report RDLC

1. Scopo generale del codice

Il file **ReportManager.vb** contiene una classe *singleton* (`NotInheritable`) che centralizza tutte le operazioni relative ai report RDLC in un'applicazione Windows Forms.
Le principali funzionalità sono:

- **Aprire** un report RDLC in un `ReportViewer` a partire da una `DataTable` o da una query SQL.
- **Aggiornare** o **resettare** il `ReportViewer` .
- **Esportare** il report corrente in formato PDF, con possibilità di aprirlo automaticamente.
- **Avviare** la finestra di stampa del `ReportViewer` .

In questo modo il resto dell'applicazione può richiamare metodi statici senza preoccuparsi della configurazione interna del viewer o della gestione dei file.

---

2. Componenti chiave e funzioni

Sezione	Metodo	Descrizione
Apri Report	`ApriReport(viewer, reportName, table, dataSourceName)`	Carica un report RDLC in un `ReportViewer` usando una `DataTable` .
	`ApriReport(viewer, reportName, query, dataSourceName, parameters)`	Esegue una query SQL, riempie una `DataTable` e chiama il metodo precedente.
Gestione Viewer	`Refresh(viewer)`	Richiama `RefreshReport()` sul viewer.
	`Clear(viewer)`	Resetta il viewer (`Reset()`).

Sezione	Metodo	Descrizione
Stampa PDF	`EsportaPdf(reportName, table, dataSourceName, apriPdf)`	Esporta il report in PDF, salva su disco e opzionalmente lo apre.
	`Stampa(viewer)`	Mostra la finestra di stampa del viewer.

Dettagli di implementazione

- **CartellaReport** – costante che indica la sottocartella “Reports” dove sono conservati i file `.rdlc`.
- **Path.Combine** – costruisce il percorso completo del report.
- **File.Exists** – verifica l’esistenza del file prima di procedere.
- **ReportViewer** – viene resettato, impostato su `ProcessingMode.Local`, e configurato con il percorso del report e la fonte dati (`ReportDataSource`).
- **Zoom e DisplayMode** – impostati su `PrintLayout` e `PageWidth` per una visualizzazione ottimale.
- **Esportazione PDF** – utilizza `LocalReport.Render("PDF")` per ottenere un array di byte, che viene poi scritto su disco con `File.WriteAllBytes`.
- **Process.Start** – apre il PDF appena creato se `apriPdf` è `True`.
- **LogLeggiScrivi.ScriviLogMsg** – chiamata a un logger interno per registrare errori e operazioni.

---

3. Flusso di esecuzione

3.1 Apertura di un report con `DataTable`

1. **Validazione**: controlla che `viewer`, `table`, `reportName` e `dataSourceName` non siano nulli o vuoti.
2. **Percorso**: costruisce il percorso completo del file RDLC.
3. **Verifica file**: lancia `FileNotFoundException` se il file non esiste.
4. **Configurazione viewer**:
   - `Reset()` → `ProcessingMode.Local` → `LocalReport.ReportPath` → `DataSources.Clear()` → `DataSources.Add(...)`.
5. **Visualizzazione**: imposta `DisplayMode.PrintLayout`, `ZoomMode.PageWidth` e chiama `RefreshReport()`.
6. **Gestione eccezioni**: registra l’errore con `LogLeggiScrivi`.

3.2 Apertura di un report con query SQL

1. **Esecuzione query**: `DB.FillDataTable(query, parameters)` restituisce una `DataTable`.

2. **Chiamata**: passa la `DataTable` al metodo di apertura precedente.
3. **Gestione eccezioni**: registra l'errore e rilancia un `Exception` con messaggio più chiaro.

3.3 Esportazione PDF

1. **Validazione**: simile al metodo di apertura.
2. **Percorso**: costruisce il percorso del report.
3. **SaveFileDialog**: chiede all'utente dove salvare il PDF.
4. **Creazione** `LocalReport`: imposta `ReportPath`, aggiunge la fonte dati e renderizza in PDF.
5. **Scrittura su disco**: `File.WriteAllBytes`.
6. **Log**: registra il percorso del file creato.
7. **Apertura PDF**: se `apriPdf` è `True`, avvia il lettore PDF predefinito.
8. **Eccezioni**: log degli errori.

3.4 Stampa

1. **Verifica**: se il viewer è `Nothing`, esce.
2. **PrintDialog**: mostra la finestra di stampa.
3. **Log**: registra l'avvio della stampa.
4. **Eccezioni**: log degli errori.

> ✏️ 🔹 **Conclusioni**
>
> Il `ReportManager` è un helper centralizzato che semplifica la gestione dei report RDLC in un'applicazione Windows Forms.
> Grazie ai metodi statici, il resto del codice può aprire, stampare o esportare report con poche righe di chiamata, lasciando a `ReportManager` la responsabilità di configurare il viewer, gestire i file e registrare gli errori.
> Effettuate piccole ottimizzazioni (gestione delle eccezioni, configurabilità, thread-safety) il componente é ancora più robusto e flessibile.